



Python 數據分析2: Pandas & Matplotlib

By Wendy Chin
On Dec.,^{7th}, 2022

Pandas簡介

Pandas 是 Python 语言的一个扩展程序库，用于数据分析。

它的名字衍生自术语 “panel data”（面板数据）和 “Python data analysis”（Python 数据分析），可以从各种文件格式比如 CSV、JSON、SQL、Microsoft Excel 导入数据，对各种数据进行运算操作，比如归并、再成形、选择，还有数据清洗和数据加工特征。未导出之前的數據都在Python緩衝内存中。

Pandas有兩種數據，分別是Series & DataFrame

- **Series:**
 - 一种类似于一维数组的对象，它由一组数据（各种Numpy数据类型）以及一组与之相关的数据标签（即索引）组成
- **DataFrame:**
 - 一个表格型的数据结构，它含有一组有序的列，每列可以是不同的值类型（数值、字符串、布尔型值）。DataFrame 既有行索引也有列索引，它可以被看做由 Series 组成的字典（共同用一个索引）。

Pandas 數據結構 - Series

Series 由索引 (index) 和列组成，函数如下：

`pandas.Series( data, index, dtype, name, copy)`

参数说明：

- data: 一组数据(ndarray 类型)。
- index: 数据索引标签，如果不指定，默认从 0 开始。
- dtype: 数据类型，默认会自己判断。
- name: 设置名称。
- copy: 拷贝数据，默认为 False

```
import pandas as pd
```

```
a=['This' 'is' 'Python']  
sr1=pd.Series(a)  
print(sr1)
```

```
0    This  
1     is  
2  Python  
dtype: object
```

index

```
a=['This' 'is' 'Python']  
sr2=pd.Series(a, index=['x' 'y' 'z'])  
print(sr2)
```

```
x    This  
y     is  
z  Python  
dtype: object
```

```
a2={1:'This' ,2:'Is' ,3:'Python'}  
sr3=pd.Series(a2, index=[1,2])  
print(sr3[1])
```

This

```
a2={1:'This' ,2:'Is' ,3:'Python'}  
sr3=pd.Series(a2, index=[1,2])  
print(sr3)
```

```
1    This  
2     Is  
dtype: object
```

Pandas 數據結構 - DataFrame

DataFrame 可以看成是 Series 組成， 函數如下：

`pandas.DataFrame( data, index, columns, dtype, copy)`

参数说明：

- **data**：一组数据(ndarray、series, map, lists, dict 等类型)。
- **index**：索引值，或者可以称为行标签。
- **columns**：列标签，默认为 RangeIndex (0, 1, 2, ..., n)。
- **dtype**：数据类型。
- **copy**：拷贝数据，默认为 False

Series 1		Series 2		Series 3		DataFrame			
Mango		Apple		Banana		Mango Apple Banana			
0	4	0	5	0	2	0	4	5	2
1	5	1	4	1	3	1	5	4	3
2	6	2	3	2	5	2	6	3	5
3	3	3	0	3	2	3	3	0	2
4	1	4	2	4	7	4	1	2	7

```
a3=[{'a':1,'b':2},{'a':3,'b':4,'c':5}]  
df1=pd.DataFrame(a3)  
print(df1)
```

		a	b	c	column
0		1	2	NaN	index
1		3	4	5.0	

沒有值，顯示NaN

DataFrame-導入、導出數據

設import pandas as pd, df為DataFrame數據變量名稱,s為Series數據變量名稱

序号	類型	名称	說明	範例
1	導入數據	pd.DataFrame()	創建空的數據框	
2		pd.read_csv(路徑+文件名)	讀取csv文件	df=pd.read_csv('D:/Demo/A1.csv')
3		pd.read_table(路徑+文件名)	從限定分隔符的文本文件導入數據	
4		pd.read_excel(路徑+文件名)	讀取excel文件	
5		pd.read_sql(路徑+文件名)	讀取sql文件	
6		pd.read_json(路徑+文件名)	讀取json文件	
7		pd.read_html(路徑+文件名)	讀取html文件,抽取其中的table	
8	導出數據	df.to_csv(路徑+文件名)	導出到指定路徑+文件名的csv文件	
9		df.to_excel(路徑+文件名)	導出到指定路徑+文件名的excel文件	
10		df.to_sql(路徑+文件名)	導出到指定路徑+文件名的sql文件	
11		df.to_json(路徑+文件名)	導出到指定路徑+文件名的json文件	
12		writer=pd.ExcelWriter(路徑+文件名) df.to_excel(writer,sheet_name=表名) writer.save()	多個數據(df,df1...)導入到同一個excel文件	

按照索引讀取表，也可以按照表名，多個表用類似[0,1]表達，如sheet_name=None,則表示讀取所有sheets,header=i即表示第i+1行設為標題列

```
path1='C:/Users/wendy_chin/Desktop/Demo/'
flNm1="VendorStock2021-Up.xlsx"

# 按照索引讀取表，也可以按照表名，多個表用類似[0,1]表達，如sheet_
wbdata1=pd.read_excel(path1+flNm1,sheet_name=0,header=5)
```

DataFrame-查看、截取數據

序号	類型	名称	說明
13	查看數據	df.head(n)	返回前n行數據，無n的時候默認5
14		df.tail(n)	返回倒數n行數據，無n的時候默認5
15		df.shape	返回數據的行數與列數
16		df.info()	返回數據的索引/數據類型/內存信息
17		df.columns	返回數據的列名
18		df.describe()	返回數值型數據的匯總
19		s.value_counts(dropna=False)	返回Series數據的唯一值與計數
20		df.isnull().any()	返回是否有缺失值
		df.isin(x)/df[col].isin(x)	返回是否為x/某列是否為x
21		df[df[column_name].duplicated()]	返回column_name字段重複數據的信息
22		df[df[column_name].duplicated()].count()	返回column_name字段重複數據的個數

序号	類型	名称	說明
23	截取數據	df[column]	返回column為列名的Series數據
24		df[[col1,col2]]	返回多列的dataframe數據
25		s.iloc[n]	按照行號n返回值
26		s.loc[row_name]	按照 行名row_name返回值
27		df.loc[n]	按照行號n返回值，但如果-1時會報錯，因為-1標籤不存在
28		df.loc[row_name]	按照行名返回值，注意，若某行索引標不是數值時，必須是名稱
29		df.iloc[n]	按照行號n返回值，可以使用-1表達最後一行，選取多行df.iloc[[x1,x2...]]
30		df.iloc[[行],[列]]	按照索引數值下標取指定行、列，其中用：進行切片，例如取所有行，列，則直接用df.iloc[:] 返回dataframe片段(參數引用中括號[])或Series(參數未引用中括號[])
31		df.loc[[行],[列]]	按照索引名稱取指定行、列，方法同上
32		df.iloc[x,y]	按照索引數值下標取第x+1行第y+1列的值，返回一個值
33		df.loc[x,col_name]	按照索引取第x+1行列名為col_name的值，返回一個值

```

a3=[{'a':1,'b':2},{'a':3,'b':4,'c':5},{'a':6,'b':7,'c':8}]
df1=pd.DataFrame(a3)
print(df1.iloc[:]);print()
print(df1.iloc[:,-1]);print()
print(df1.iloc[0:-1]);print()
print(df1.iloc[:, -1]);print()
print(df1.iloc[[0,1],[1,2]]);print()
print(df1.iloc[0,2]);print()
  
```

1

```

a  b  c
0  1  2  NaN
1  3  4  5
2  6  7  8
  
```

2

```

0  NaN
1  5
2  8
Name: c, dtype: object
  
```

3

```

0  NaN
Name: c, dtype: object
  
```

4

```

c
0  NaN
1  5
2  8
  
```

5

```

b  c
0  2  NaN
1  4  5
  
```

6

```

nan
  
```


DataFrame-數據處理、排序、匯總、篩選、統計

序	類型	名称	說明
34	數據處理	df.columns[col1,col2,col3...]	重新命名列名, 全全部羅列出來, 否則會報錯
35		df.notnull()	檢查非空值, 並返回布爾值
36		df.dropna()	刪除所有含空值的行
37		df.dropna(axis=1)	刪除所有含空值的列
38		df.fillna(value=x)/df.fillna(x)	用x取代所有的空值
39		s.astype(float)	將數據類型轉為float型
40		s.replace(['x','y'],[1,'2'])	將x更換為1, y更換為2
41		df.rename(columns={'colx':'coly'},inplace=True)	更換某列的名稱colx為coly, 其中inplace=True 必須注明, 否則就不會真正修改原對象
42		df.rename(index={'x':'y'},inplace=True)	更換某行的名稱x為y, 其中inplace=True 必須注明, 否則就不會真正修改原對象
43		df.rename_axis(x,axis=1)	將columns重新命名, 黨x=[None],則表示去除column
44		df.set_index(x)	將某個字段x設為index
45		df.reset_index(drop=True,inplace=True)	對行(index)重新排序,drop=True表示去掉原數據
46	數據統計	df.drop(index=[1],columns=[[0,2,3]])	刪除某行某些列
47		df.drop_duplicates(subset=col1,keep='first',inplace=True)	刪除某列重複值, keep=['first','last',False] 表示保留第一個/最後一個/保留所有. inplace=True表示从原始数据删除
48		df[new_col]=[]	新增某列new_col并进行赋值

序号	類型	名称	說明
47	數據排序	df.sort_index()	按照索引排序, 默認升序
48		df.sort_values()	按照索引排序, 默認升序 若參數為False,則為降序
49	數據匯總	df.groupby(col1)	返回一個按照列名為col1分組的對象
50		df.groupby([col1,col2])	返回一個按照列名為col1, col2分組的對象
51		df.groupby(col1)[col2].count()	按照col1統計col2的個數
52		df.groupby(col1)[col2].mean()/sum()	按照col1統計col2的總和, 平均值
53	數據篩選	df.pivot_table(value=[col1,col2...],index=[colx,coly..],aggfunc='sum',fill_value=0,margins=True,margins_name=x)	樞紐分析表, 其中選用col1,col2...列的值為分析內容, colx,coly 為列分析字段索引, 匯總方式為求和, 空白值為0, 匯總顯示名字為x
54		df.query('city==['SZ','SH'])	查詢city列等於SZ, SH的數據
55		df.loc[(df['aging']>90)&(df['Sloc']=='A1')]	查詢aging列>90且Sloc列等於A1的數據
56	數據統計	df.loc[(df['aging']>90) (df['Sloc']=='A1')]	查詢aging列>90或Sloc列等於A1的數據
57		df.sample(n=x,weights=[y1,y2..],replace=False)	設置採樣數為x, 權重[y1,y2...],取樣後數據不放回的樣本
58		df[col1].std()	計算某列col1的方差
59		df[col1].cov(df[col2])	計算某列col1與col2的協方差
60		df[col1].corr(df[col2])	計算某列col1與col2的相關性分析

```
a3=[{'a':'1','b':2},{'a':'3','b':4,'c':5},{'a':'6','b':7,'c':8}]
df1=pd.DataFrame(a3)
...
df1.rename(columns={'b':'x'},inplace=True)
print(df1)
```



	a	x	c
0	1	2	NaN
1	3	4	5

DataFrame-數據類型轉換

- 轉日期時間格式

- `pd.to_datetime()`

`pd.to_datetime(df['On shift'], format="%H:%M:%S")`

- 字符串轉數值

- `pd.to_numeric(errors="")`

例如:`pd.to_numeric(df['On shift'])`

注意: 如果字符串含有字母, 無法轉
`errors`為`raise`時, 遇到無法轉時, 報錯
`errors`為`coerce`時, 遇到錯誤, 用`NaN`取代

- 其他類型轉字符串/數值/布爾

- `df[x].astype()`

例如:`df['On shift'].astype(str)`

`astype`參數:`str`, `float`, `int`, `complex`, `bool` & `numpy`支持的其他`dtype`

```
path1='C:/Users/wendy_chin/Desktop/Demo/'
flnm1='shifttime.xlsx'

df=pd.read_excel(path1+flnm1,sheet_name=0)
df['New1 On shift']=pd.to_datetime(df['On shift'],format='%H:%M:%S')
df['New2 On shift']=pd.to_numeric(df['On shift'],errors='coerce')
df['New3 On shift']=df['On shift'].astype(str)
print(df[['On shift','New1 On shift','New2 On shift','New3 On shift']])
```

`pd.to_datetime()`常用的參數是`format`, 按照指定的字符串`strftime`格式解析日期, 一般情況下該函數可以直接自動解析成日期類[之後可得`series.dt`屬性(如`year/month/date/hour/date/time`等)]

`astype()`可強制轉換數據類型, 特別注意, 可能會導致某些數據轉換後失效。

	On shift	New1 On shift	New2 On shift	New3 On shift
0	07:00:00	1900-01-01 07:00:00	NaN	07:00:00
1	07:00:00	1900-01-01 07:00:00	NaN	07:00:00
2	07:00:00	1900-01-01 07:00:00	NaN	07:00:00
3	07:00:00	1900-01-01 07:00:00	NaN	07:00:00
4	07:00:00	1900-01-01 07:00:00	NaN	07:00:00

DataFrame-数据合并

序	類型	名称	說明
49	数据合并	<code>df1.concat(df2)/pd.concat(df1,df2,axis=0/1,join='inner'/'outer')</code>	数据合并,axis=0表示df2添加在df1的行之后, axis=1表示添加在列之后, 默认为0; join为outer表示合集, 空值用NaN表示; 为inner表示只合并相同的字段属性, 默认为outer
50		<code>df1.merge(df2)/pd.merge(df1,df2,how='inner'/'outer'/'right'/'left',on=x,left_on="",right_on="",sort=False,suffixes=("_x","_y"))</code>	数据合并,默认how=inner, 通常取交集;how=outer表示保留所有数据, 空值为NaN;'right'表示保留df2,'left'表示保留df1, 其中on="是关键索引;suffixes为合并后新生成的df列名后缀

```
DataFrame.merge(left, right,
                 how='inner',    # {'left', 'right', 'outer', 'inner'}, default 'inner'
                 on=None,
                 left_on=None, right_on=None,
                 sort=False,
                 suffixes=('_x', '_y'))
```

DataFrame-merge()詳解

序号	類型	名称	說明	範例
			left:表示第一個DataFrame/Series數據 right:表示第二個DataFrame/Series數據 how:默認inner,{'left', 'right', 'outer', 'inner', 'cross'}, left表示保留第一個df的順序;right表示保留第二個df的順序, outer表示第一個&第二個的合集, 默認字典式排序,inner表示兩者交集, 以第一個為主順序, cross表示產生笛卡爾式數據, 已第一個為主順序。 on:表示關鍵索引, 即兩個df都有的列名或者行索引,不可與left_on/right_on同時用 left_on:第一個df的列名或者行索引, 與 right_on:第二個df的列名或者行索引, 同上 left_index:默認為False,表示是否以第一個df作為索引, 若是, 則兩個df的索引數要一樣 right_index:默認為False表示是否以第二個df作為索引, 若是, 則兩個df的索引數要一樣 sort:默認False, 表示是否排序, 若是, 則按照相關的索引設定字典式排序 suffixes:非關鍵列名相同時默認生成加"_x"/"_y"為後綴的列名 copy:默認True, 表示否可複製 indicator:默認False,表示是否顯示merge的相關細節信息, 若是, 則會增加一列"_merge"在每一行顯示merge信息 validate:檢驗是否符合merge的數據類型, ("1:1","1:m","m:1","m:m"), 1:1表示兩個df的字段都是唯一的,1:m表示第一個df的索引是唯一的	<pre>df1 = pd.DataFrame({'lkey': ['baz', 'bar', 'foo', 'foo'], 'value': [1, 2, 3, 4]}) df2 = pd.DataFrame({'rkey': ['bar', 'foo', 'baz', 'foo', 'xyz'], 'value': [5, 6, 7, 8, 9]}) df3=pd.merge(df1,df2,left_on='lkey',right_on='rkey')</pre>

	lkey	value
0	baz	1
1	bar	2
2	foo	3
3	foo	4

df1

	rkey	value
0	bar	5
1	foo	6
2	baz	7
3	foo	8
4	xyz	9

df2

	lkey	value_x	rkey	value_y
0	baz	1	baz	7
1	bar	2	bar	5
2	foo	3	foo	6
3	foo	3	foo	8
4	foo	4	foo	6
5	foo	4	foo	8

df3

```
df1 = pd.DataFrame({'lkey': ['baz', 'bar', 'foo', 'foo'], 'value': [1, 2, 3, 4]})
df2 = pd.DataFrame({'rkey': ['bar', 'foo', 'baz', 'foo', 'xyz'], 'value': [5, 6, 7, 8, 9]})
df3=pd.merge(df1,df2,left_on='lkey',right_on='rkey')
```

DataFrame-merge()範例

```
path1='C:/Users/wendy_chin/Desktop/Demo/'  
flNm1="VendorStock2021-Up.xlsx"
```

按照索引讀取表，也可以按照表名，多個表用類似[0,1]表達，如sheet_name=None，則表示讀取所有sheets，header=i即表示第i+1行設為標題列

```
data1=pd.read_excel(path1+flNm1,sheet_name=0,header=5)
```

刪除多行類似index[[0,1,2]]、多列類似columns[[0,1,2]]后，對行(index)重新排序 (drop=true表示去掉原有的數據標籤)

```
data1=data1.drop(index=data1.index[[0]],columns=data1.columns[[0,2,3]])
```

```
data1.reset_index(drop=True,inplace=True)
```

```
data2=pd.read_excel(path1+flNm1,sheet_name=1,header=1)
```

```
data2=data2.drop(columns=data2.columns[0])
```

設定how='left'表示data1 index排序不變，否則不設定how則index會重新排序

```
data3=pd.merge(data1,data2,left_on='StorageBin',right_on='S.L',how='left')
```

```
# data3=data3.loc[:,['Material','Plnt','StorageBin','Actual']] #取某些列名所在的所有index數據
```

	Material	Plnt	S	...	Typ	BUn	GR Date
0	A0N410M1	3AX*	NaN	...	399.0	PC	2022-11-16 00:00:00
1	AM-2M11L	3AX*	NaN	...	399.0	PC	2022-06-13 00:00:00
2	AM-2N41L	3AX*	NaN	...	399.0	PC	2022-07-08 00:00:00
3	AM-2N420	3AX*	NaN	...	399.0	PC	2022-09-11 00:00:00
4	AM-2N420	3AX*	NaN	...	399.0	PC	2022-09-25 00:00:00
...	...	...	...	...	...	...	...
195	M0EL4N10	3AX*	Q	...	399.0	PC	2023-02-21 00:00:00
196	M0EL4N10	3AX*	Q	...	399.0	PC	2023-02-09 00:00:00
197	M0NLL10	3AX*	Q	...	399.0	PC	2023-02-24 00:00:00
198	M0NLL10	3AX*	NaN	...	399.0	PC	2023-02-24 00:00:00
199	M0NLLH10	3AX*	Q	...	399.0	PC	2022-10-19 00:00:00

[200 rows x 11 columns]

data1

Merge



	S.L	Actual	Site
0	A1	N1	SZ
1	B2	N2	SZ
2	C3	N3	SH
3	D4	N4	SH
4	E5	N5	SZ
5	F6	N6	SH

data2



	Material	Plnt	S	StorageBin	...	GR Date	S.L	Actual	Site
0	A0N410M1	3AX*	NaN	A1	...	2022-11-16 00:00:00	A1	N1	SZ
1	AM-2M11L	3AX*	NaN	A1	...	2022-06-13 00:00:00	A1	N1	SZ
2	AM-2N41L	3AX*	NaN	A1	...	2022-07-08 00:00:00	A1	N1	SZ
3	AM-2N420	3AX*	NaN	A1	...	2022-09-11 00:00:00	A1	N1	SZ
4	AM-2N420	3AX*	NaN	A1	...	2022-09-25 00:00:00	A1	N1	SZ
...	...	...	...	...	...	...	...	...	...
195	M0EL4N10	3AX*	Q	C3	...	2023-02-21 00:00:00	C3	N3	SH
196	M0EL4N10	3AX*	Q	C3	...	2023-02-09 00:00:00	C3	N3	SH
197	M0NLL10	3AX*	Q	A1	...	2023-02-24 00:00:00	A1	N1	SZ
198	M0NLL10	3AX*	NaN	B2	...	2023-02-24 00:00:00	B2	N2	SZ
199	M0NLLH10	3AX*	Q	A1	...	2022-10-19 00:00:00	A1	N1	SZ

data3

[200 rows x 14 columns]

DataFrame-pivot_table()詳解

序号	類型	名称	說明	範例
	樞紐分析表		None:表示可以放入所需的數據 data:表示放入DataFrame類型的數據 values:需要計算的列，相當於excel樞紐分析表中的“值” Index: 需要作為行索引的列，相當於excel樞紐分析表中的“列” columns:需要作為列索引的列，相當於excel樞紐分析表中的“欄” aggfunc: 計算方法，例如sum/mean/max/min/np.count_nonzero (加總/平均/最大/最小/計數) fill_value: 表示出現空缺值時對其進行賦值 margins:默認False,表示是否顯示小計/總計 dropna:默認true,表示是否刪除空缺值 margins_name:小計/總計名稱 observed:適用於Categoricals, 默認False sort:默認True, 表示是否排序	<pre>pd.pivot_table(df,values=['D','E'],index=['A','B'],columns=['C'],aggfunc={'D':'sum','E':np.count_nonzero},fill_value=0,margins=True,margins_name="Total")</pre>

```
df = pd.DataFrame({"A": ["foo", "foo", "foo", "foo", "foo", "bar", "bar", "bar", "bar", "bar"],
                   "B": ["one", "one", "one", "two", "two", "one", "one", "two", "two", "two"],
                   "C": ["small", "large", "large", "small", "small", "small", "large", "small", "small", "large"],
                   "D": [1, 2, 2, 3, 3, 4, 5, 6, 7],
                   "E": [2, 4, 5, 5, 6, 6, 8, 9, 9]})
```

	A	B	C	D	E
0	foo	one	small	1	2
1	foo	one	large	2	4
2	foo	one	large	2	5
3	foo	two	small	3	5
4	foo	two	small	3	6
5	bar	one	large	4	6
6	bar	one	small	5	8
7	bar	two	small	6	9
8	bar	two	large	7	9

```
pvtble1=pd.pivot_table(df,values=['D','E'],index=['A','B'],columns=['C'],aggfunc={'D':'sum','E':np.count_nonzero},fill_value=0,margins=True, margins_name="Total")
```

樞紐分析表欄位

選擇要新增到報表的欄位:

搜尋

1

data

在下列區域之間拖曳欄位:

篩選

欄

列

Σ 值

3

index

4

columns

2

values

更新

		D			E		
C		large	small	Total	large	small	Total
A	B						
bar	one	4	5	9	1	1	2
	two	7	6	13	1	1	2
foo	one	4	1	5	2	1	3
	two	0	6	6	0	2	2
Total		15	18	33	4	5	9

DataFrame-pivot_table範例

```
path1='C:/Users/wendy_chin/Desktop/Demo/'
flNm1="VendorStock2021-Up.xlsx"

# 按照索引讀取表，也可以按照表名，多個表用類似[0,1]表達，如sheet_name=None，則表示讀取所有sheets，header=i即表示第i+1行設為標題列
wbdata1=pd.read_excel(path1+flNm1,sheet_name=0,header=5)
# print(wbdata1)

# 刪除多行類似index[[0,1,2]]、多列類似columns[[0,1,2]]后，對行(index)重新排序（drop=true表示去掉原有的數據標籤）
wbdata1=wbdata1.drop(index=wbdata1.index[[0]],columns=wbdata1.columns[[0,2,3]])
wbdata1.reset_index(drop=True,inplace=True)

# 產生樞紐分析表
pivottable1=pd.pivot_table(wbdata1,values=[wbdata1.columns[4],wbdata1.columns[8]],
                           index=[wbdata1.columns[3],wbdata1.columns[0]],aggfunc='sum',
                           fill_value=0,margins=True,margins_name='Total')
# print(pivottable1)

pivottable2=pd.pivot_table(wbdata1,values=['Available stock','Typ'],index=['Material'],
                           columns=['StorageBin'],aggfunc={'Available stock':'sum','Typ':np.count_nonzero},
                           fill_value=0,margins=True,margins_name='Total')

print(pivottable2)
pivottable2.columns=pivottable2.columns.droplevel(0)
# pivottable1.index=pivottable1.index.droplevel(0)
# 去除columns名称并重新排序，则会显示pivot中index重复值
pivottable1=pivottable1.rename_axis([None],axis=1).reset_index()
print(pivottable2)
```

#去除第一層列名称，但總層數要大於1，否則報錯

#去除第一層行名称，但總層數要大於1，否則報錯

DataFrame-范例1: 拆分表格成不同文件

```
path1="D:\\RPA Demo\\Excel\\"
fileNm1="VendorStock2021-Up.xlsx"
path2='C:/Users/wendy_chin/Desktop/Demo/Temp/'
tday=datetime.datetime.today()

df=pd.read_excel(path1+fileNm1,sheet_name=None,header=None)
# sheet_name=None表示读所有的工作表，得到一个字典，关键字一般为工作表名；header=None表示不将第一行设为列标题
for key in df.keys():
    if 'Stock' in key:
        df1=df[key]
        dfnew=df1.drop(columns=df1.columns[[0,2,3]],axis=1)
        dfnew.columns = list(dfnew.iloc[5]) #设置第6行为列标题
        dfnew=dfnew.drop(index=df1.index[[0,1,2,3,4,5,6]])
        dfnew.reset_index(drop=True,inplace=True)
        dfnew[dfnew['GR Date']=='2022/2/29']=np.NaN
        # dfnew['GR Date'].fillna('2022-02-28 00:00:00')
        dfnew['aging']=tday-pd.to_datetime(dfnew['GR Date']) # Pandas 同类型日期格式可加减

groups=dfnew.groupby(by=['StorageBin'])
for item,grp in groups:
    newFL=str(item)+'.xlsx'
    grp.to_excel(path2+newFL,sheet_name='detail',index=False) # index=False表示不将Dataframe中的index写入
```

DataFrame-范例2: 多个文件汇总成一个文件

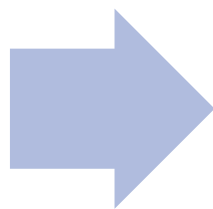
```
df_new=pd.DataFrame()
for fullname in os.listdir(path2):
    filename=fullname.split('.')[0]
    if filename[0].isalpha() and len(filename)==2:
        df_1=pd.read_excel(path2+fullname,sheet_name='detail')
        df_new=pd.concat([df_new,df_1],axis=0)

df_new.to_excel(path1+'VS.xlsx',sheet_name='All',index=False)
```

Matplotlib-簡介

Matplotlib 是一个非常强大的 Python 画图工具，我们可以使用该工具将很多数据通过图表的形式更直观的呈现出来,可以绘制线图、散点图、等高线图、条形图、柱状图、3D 图形、甚至是图形动画等等

其中Pyplot 是 Matplotlib 的子库，提供了和 MATLAB 类似的绘图 API。包含一系列绘图函数的相关函数，每个函数会对当前的图像进行一些修改，例如：给图像加上标记，生新的图像，在图像中产生新的绘图区域等等。



使用的时候，我们可以使用 import 导入 pyplot 库，并设置一个别名 **plt**

```
import matplotlib.pyplot as plt  
import matplotlib.animation as animation
```

Matplotlib-常用設置

序号	類型	內容	說明	範例
1	創建窗口	<code>plt.figure(figsize=)</code>	創建一個繪圖窗口並定義大小(英寸)	<code>plt.figure(figsize=(10,4))</code>
2	參數設置	<code>plt.rcParams[]=[]</code>	設置默認字體或坐標負值顯示等	<code>plt.rcParams['font.sans-serif']=['Calibri']</code> <code>plt.rcParams['axes.unicode_minus']=False</code>
3	柱狀圖	<code>plt.bar(x,y,width=,align=,color=,label=)</code>	<code>align(center,edge),</code> <code>width</code> :相對寬度,默認0.8 <code>color(r,g,b,c,m,y,k,w)</code> 等 <code>label</code> : 圖標籤	<code>plt.bar(x,y,color='blue',edgecolor='green',label='Location')</code>
4	折綫圖	<code>plt.plot(x,y,linewidth=,linestyle=,marker=,markersize=)</code>	<code>linestyle:solid,dashed,dotted,dashdot,none</code> <code>marker:','o','v','s','*','p','h','D','d','+','x'</code>	<code>plt.plot(x,y,linewidth=3,linestyle='solid',marker='s',markersize=10)</code>
5	餅圖	<code>plt.pie(y,labels=x,labeldistance=,autopct=,pctdistance=,counterclock=False,startangle=)</code>	<code>labeldistance</code> :餅塊數據標籤與中心的距離 <code>autopct</code> :設置百分比格式 <code>pctdistance</code> :百分比標籤與中心的距離 <code>counterclock:False</code> 為順時針 <code>startangle</code> :第一個餅塊的初始角度	<code>plt.pie(y,labels=x,labeldistance=1.1,autopct='%0.2f%%',pctdistance=1.5,counterclock=False,startangle=90)</code>
6	設置圖例	<code>plt.legend(loc=)</code>	前提是必須在相對的圖形類型中有 <code>label</code> 的屬性定義才會顯示 <code>legend</code> <code>loc:best,upper right,upper left,lower left,lower right,right,center left,center right,lower center,upper center,center</code>	<code>plt.legend(loc='best')</code>
7	圖標題	<code>plt.title(loc=)</code>	<code>loc:center, left,right</code>	<code>plt.title(label='VS Summary',fontdict={'family':'Calibri','color':'blue','size':16},loc='left')</code>
8	數據標籤	<code>plt.text(x=a,y=b,s=b,ha=,va=)</code>	<code>x=a</code> 表示設置x坐標的標籤值為a <code>y=b</code> 表示設置y坐標的標籤值為b <code>s=b</code> 表示顯示文本內容為b <code>ha</code> :表示水平對齊方式(<code>center, left,right</code>) <code>va</code> :表示垂直對齊方式(<code>center, top,bottom,baseline,center_baseline</code>)	

Matplotlib-範例

```
#对以上结果进行画图
pic=plt.figure()      #创建一个画布， 若只产生一个图可忽略
plt.rcParams['font.sans-serif']=['Calibri']      #设置默认字体
plt.rcParams['axes.unicode_minus']=False      #解决坐标值未负数无法显示负号的问题
x=pivottable['StorageBin']
y=pivottable['    Available stock']

plt.bar(x,y,color='blue',edgecolor='green',label='Location')      # color 为柱子填充色，edgecolor：柱子边框颜色
plt.title(label='VS Summary',fontdict={'family': 'Calibri', 'color': 'blue', 'size': 16},loc='left')      #标题内容以及格式
plt.xlabel('location',fontdict={'family': 'Calibri', 'color': 'blue', 'size': 12},labelpad=10)      #x轴标题以及格式
plt.legend(loc='upper center',fontsize=10)      #设置图例位置与字体大小,本例中需要在bar中有label才有效，否则无法产生图例
for a,b in zip(x,y):
    plt.text(a,b,b,fontdict={'family': 'Calibri', 'color': 'blue', 'size': 10})      #设置数据标签

# plt.show()      #显示图表

pic2=plt.figure()
```


課後作業

1. Python代碼：操作<shifttime.xlsx>找出異常出勤的記錄
2. Python代碼：操作<VendorStock-Up.xlsx>文件：
 - 2.1) 查找含有“Stock”的表名，
 - 2.2) 找到Item 2.1的工作表后，查找距当前日期呆滯天数超过180天的清单
 - 2.3) 找到Item 2.2的数据后，以“StorageBin”的地点代码拆分成不同文件，
文件名称以“StorageBin”下的地点命名.xlsx，例如“A1.xlsx”
3. 根据作业2，将拆分后的文件再合并成新文件“VS.xlsx”
4. 以“StorageBin”的地址碼為x軸, available qty為y軸產生一個柱狀圖，

Thank You